PromptFunds

TRD

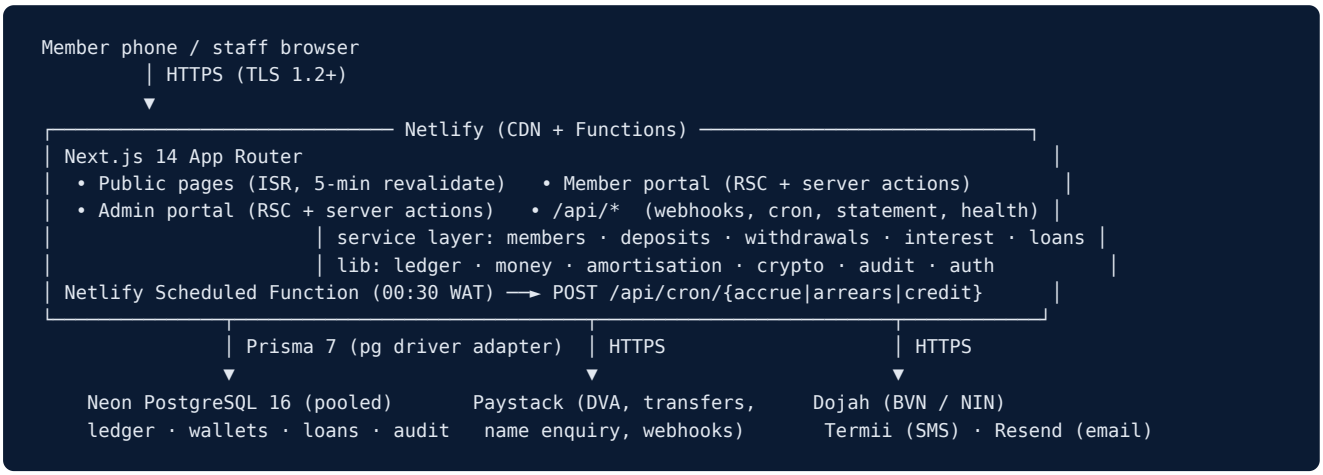
Technical Requirements Document

Prompt Funds — architecture, stack, security and operations

VERSION	DATE	PREPARED FOR	CLASSIFICATION
1	November 2026	Prompt Funds / cooperative lending clients	Confidential

1. Architecture overview

Prompt Funds is a **modular monolith**: one Next.js application containing the public site, member portal, admin portal and API, backed by one PostgreSQL database. Business logic lives in a service layer that both the UI (server actions) and the HTTP API call. This keeps a small team fast and keeps money movements inside single ACID transactions. Services are separated by module so they can be split out later if volume demands it.



1.1 Why this stack

Layer	Choice	Reason
UI + server	**Next.js 14 (React 18, TypeScript)**	One codebase for SEO-friendly public pages, fast server-rendered dashboards and APIs; server components keep JavaScript on phones small
Styling	**Tailwind CSS**	Small CSS output, consistent design tokens, fast to build mobile-first layouts
Database	**PostgreSQL 16** (Neon serverless in cloud)	ACID transactions, row locks, CHECK constraints, triggers, `pg_trgm` search — essential for a ledger
Data access	**Prisma 7** with `@prisma/adapter-pg`	Type-safe queries and migrations; the Rust-free client has no native binaries, so it bundles cleanly into Netlify/Lambda functions with smaller cold starts
Raw SQL	Ledger constraints, triggers, reporting aggregates	Integrity rules enforced by the database itself, not just the app
Validation	**Zod**	Every input validated at the edge of the system
Payments	**Paystack** (Dedicated Virtual Accounts, Transfers, Resolve Account) behind an adapter; Monnify/Flutterwave as alternatives	Each member gets a permanent account number so deposits reconcile automatically
Identity	**Dojah** behind an adapter; Prembly, Monnify, Smile ID as alternatives	BVN/NIN verification against government sources
Hosting	**Netlify** (primary) or any container host (Dockerfile provided)	Git-push deploys, previews per branch, scheduled functions
Monitoring	Sentry (errors), Better Stack / UptimeRobot (uptime), Netlify logs	Fast detection of failures in money flows

Layer	Choice	Reason
CI	GitHub Actions: typecheck, unit tests, build	Every change verified before deploy

Where C# / .NET and SQL fit

- **SQL** is used directly where it is the right tool: append-only and balance triggers on the ledger, CHECK constraints on wallets, trigram search indexes, and set-based reporting (trial balance, portfolio ageing).
- **C#/.NET is not recommended for the launch build.** Splitting the stack would double hiring, CI and deployment cost for no functional gain at this volume. It becomes sensible later for a separate **reconciliation and regulatory-reporting worker** (e.g. .NET 8 worker service reading settlement files and producing XLSX/XBRL returns) if the cooperative's IT team is .NET-skilled. The service boundaries and database design allow that without changing the web app.

2. Code structure

```
prisma/
  schema.prisma          data model (money = BigInt kobo)
  migrations/            SQL migrations incl. ledger triggers + search indexes
  seed.ts                idempotent: chart of accounts, staff, policies, products, demo data
prisma.config.ts         migration connection (DIRECT_URL)
netlify.toml             build + Next.js runtime plugin
netlify/functions/cron.mts   nightly scheduler → /api/cron/*
src/
  app/(public)/          home, savings, loans-info, about, faq, login, register
  app/(member)/          dashboard, kyc, wallet, transactions, loans, loans/apply, loans/[id], profile
  app/admin/             overview, members, withdrawals, loans, loans/[id], reports, operations, audit
  app/api/               auth, webhooks/paystack, webhooks/payments, cron/[job], statement, wallet,
transactions, withdrawals, health
  app/_actions/          server actions (all mutations from forms)
  services/              members, deposits, withdrawals, interest, loans, notify
  lib/                   db, ledger, money, interest, amortisation, password, crypto, audit, auth, policy,
providers/*
  components/            UI primitives, logo, loan calculator
```

3. Money and ledger design

3.1 Representation

- All amounts are **integer kobo** in `BIGINT` / JavaScript `bigint`. Floats are never used for money.
- Rates are **basis points** (`2400` = 24.00%).
- Daily interest accrues in **micro-kobo** (kobo × 10^6); conversion to kobo uses **banker's rounding** (half-to-even) at credit time.
- Loan schedules use 10^{12} fixed-point for the annuity factor; the final instalment absorbs rounding so principal sums exactly.

3.2 Double-entry ledger

Every money event is a `LedgerEntry` with ≥ 2 `LedgerLine`s whose debits equal credits. Enforcement is layered:

1. Application: `postEntry()` rejects unbalanced or zero lines and unknown accounts.

2. Database: a **deferred constraint trigger** re-checks balance at commit; **append-only triggers** reject UPDATE/DELETE on ledger tables. Corrections are reversal entries.
3. Idempotency: each entry has a unique `idempotencyKey` (e.g. `deposit:<providerRef>`), so retries cannot double-post.

Chart of accounts:

Code	Account	Type
1000-BANK-COLL	Bank – Collections (member deposits)	Asset
1010-BANK-PAYOUT	Bank – Payouts (withdrawals & disbursements)	Asset
1200-LOANS-REC	Loans receivable	Asset
2000-MEMBER-{id}	Member savings (one sub-account per member)	Liability
4000-LOAN-INT	Loan interest income	Income
4100-FEE-INC	Loan processing fee income	Income
5000-INT-SAVINGS	Interest expense on savings	Expense
5100-LOSS-PROV	Loan loss provision	Expense

Postings:

Event	Debit	Credit
Deposit confirmed	Bank-Collections	Member savings
Withdrawal paid	Member savings	Bank-Payouts
Interest credited	Interest expense	Member savings
Loan disbursed	Loans receivable (principal)	Bank-Payouts (net) + Fee income (fee)
Repayment from savings	Member savings	Loans receivable (principal part) + Loan interest income (interest part)
Disbursement failed	Bank-Payouts + Fee income	Loans receivable (reversal)

3.3 Wallet cache

`Wallet` holds cached `balanceKobo`, `heldKobo`, totals and a `version`. It is updated **in the same transaction** as the ledger posting. A nightly check compares each wallet with its ledger sub-account; any difference raises an alert. A CHECK constraint guarantees `0 ≤ held ≤ balance`.

3.4 Concurrency

- Withdrawals and repayments take `SELECT … FOR UPDATE` on the wallet row, so two parallel requests cannot overdraw.
- Withdrawal requests and repayments carry client idempotency keys (UUID in the form) — a double-tap returns the first result.
- Audit writes take a transaction-scoped advisory lock so the hash chain cannot fork.

4. Integrations

4.1 Payment partner (adapter: `lib/providers/payments.ts`)

Operation	Paystack endpoint	Notes
Create customer + dedicated account	`POST /customer`, `POST /dedicated_account`	On KYC approval
Name enquiry	`GET /bank/resolve`	Own-account check at KYC
Create transfer recipient	`POST /transferrecipient`	Cached on the member's bank account
Payout	`POST /transfer` with reference `wd_<id>` / `ln_<id>`	Only to recipient of the verified own account
Webhooks	`charge.success` (dedicated_nuban), `transfer.success/failed/reversed`	HMAC-SHA512 signature; deposits re-verified via `GET /transaction/verify/:ref`

The adapter deliberately exposes **no generic transfer function**. Switching to Monnify or Flutterwave means implementing the same interface.

4.2 Identity (adapter: `lib/providers/kyc.ts`)

BVN lookup with first name, last name and DOB match. The result score and provider reference are stored; raw responses are not. NIN, selfie liveness and document checks plug into the same interface in Phase 2.

4.3 Notifications

`services/notify.ts` writes in-app notifications inside the business transaction. Outbound SMS (Termii) and email (Resend) are dispatched **after commit** by a queue (Phase 2: Netlify background function or Inngest) so external failures never roll back money movements.

5. Scheduled jobs

Job	When (WAT)	What	Idempotency
`accrue`	Daily 00:30	Accrue interest for every wallet, in 500-row keyset batches	Unique (member, date)
`arrears`	Daily 00:30	Auto-debit due instalments from available savings; mark overdue; set arrears	Repayment key `auto-<date>`
`credit`	Last day of month	Post accrued interest to ledger	Entry key includes first accrual id + count
`reconcile`	Daily 06:00 (Phase 2)	Match provider settlement report to ledger	Run per date

Jobs are triggered by a Netlify Scheduled Function calling `/api/cron/<job>` with a bearer secret (constant-time compared). They can also run from the admin Operations page or the CLI (`npm run job:accrue`).

6. Security requirements

Control	Implementation
Transport	HTTPS only; HSTS (2 years); TLS to database and providers
Passwords	scrypt ($N=2^{15}$, r=8, p=1, 64-byte key, per-user salt); min 10 chars; lockout for 15 min after 5 failures
Sessions	256-bit random token; only its SHA-256 stored; httpOnly, Secure, SameSite=Lax cookie; 8-hour expiry; server-side revocation on logout
Staff MFA	TOTP required for all staff roles before go-live (schema field ready)
Authorisation	Role checks inside every service function (not only in the UI); maker-checker on KYC→wallet, withdrawals, loan approval and disbursement
Sensitive data	BVN/NIN encrypted with AES-256-GCM (random IV, auth tag, versioned format); HMAC-SHA256 hash for duplicate detection; masked in UI
Input	Zod schemas on every action/route; Prisma parameterised queries; tagged-template raw SQL only
Webhooks	Signature verification with constant-time compare; re-verification with provider API; idempotent handlers
Headers	X-Frame-Options DENY, nosniff, strict referrer policy, HSTS; add CSP before go-live
Secrets	Netlify environment variables (encrypted); never in git; rotate on staff exit
Audit	Hash-chained audit log for all state changes; append-only ledger
Rate limiting	Upstash Redis rate limiter on login, register, KYC and webhook routes (Phase 1 hardening)
Testing	Unit tests for money/interest/schedule/crypto; integration tests for ledger flows; external penetration test before launch

7. Performance, memory and searchability

- **Server components** render dashboards on the server; member pages ship minimal client JS (only the calculator, nav and submit buttons are client components).
- **Keyset pagination** (cursor on `createdAt, id`) for history and statements — constant cost per page regardless of history length.
- **Streaming CSV statements** using `ReadableStream`, fetching 500 rows at a time, so a large statement never loads fully into memory.
- **Batch jobs** iterate wallets in 500-row keyset chunks with `createMany … skipDuplicates`, keeping memory flat at 100k+ members.
- **Indexes** on every foreign key and query path: `(memberId, createdAt DESC)` on transactions, `(dueDate, status)` on instalments, `status` on withdrawals/loans, GIN **trigram** index on `Member.searchText` for fast fuzzy admin search by name, phone, email or membership number.
- **Connection pooling**: app uses Neon's pooled endpoint; pool size per function instance capped (default 5); migrations use the direct endpoint.
- **Caching**: public product pages use incremental static regeneration (5 min); money pages are always dynamic.

- **SEO**: server-rendered public pages, per-page titles/descriptions, semantic HTML, fast LCP; add sitemap.xml and Open Graph images in Phase 3.

8. Environments and deployment

Environment	Hosting	Data	Providers
Local	`npm run dev` + local Postgres / Docker	Seeded demo	Mock
Preview	Netlify deploy preview per pull request	Neon branch database	Mock / sandbox
Staging	Netlify (staging site)	Neon staging	Paystack test keys, Dojah sandbox
Production	Netlify (production site)	Neon production (Launch/Scale plan, PITR)	Live keys

Deploy pipeline: push → GitHub Actions (typecheck, tests, build) → Netlify build runs `prisma migrate deploy`, idempotent seed, `next build` → atomic publish. Rollback = redeploy previous build (database migrations are forward-only and backward compatible).

Environment variables: `DATABASE_URL` (pooled), `DIRECT_URL`, `FIELD_ENCRYPTION_KEY`, `IDENTITY_HASH_SECRET`, `CRON_SECRET`, `PAYMENT_PROVIDER`, `PAYSTACK_SECRET_KEY`, `PAYSTACK_DVA_BANK`, `KYC_PROVIDER`, `DOJAH_APP_ID`, `DOJAH_SECRET_KEY`, `KYC_AUTO_APPROVE`, `DEMO_MODE`, `SEED_ADMIN_PASSWORD`.

Portability: a multi-stage `Dockerfile` and `docker-compose.yml` allow running on Render, Railway, Fly.io, AWS ECS or Azure Container Apps with any managed PostgreSQL.

9. Backup, recovery and continuity

- Neon point-in-time restore (7 days on Launch, up to 30 days on Scale) plus a nightly logical dump to separate object storage retained 90 days.
- RPO ≤ 5 minutes, RTO ≤ 4 hours for production.
- Quarterly restore drill to a branch database, verified by running the trial balance.
- Provider outage: deposits queue at the provider and arrive by webhook retry; withdrawals stay in the queue; the UI shows a status banner.

10. Observability

- Structured logs for every webhook, payout and job with correlation IDs.
- Alerts: trial balance ≠ 0; wallet vs ledger mismatch; webhook signature failures > 5/hour; payout failures; job not run by 02:00; unreconciled items > 2 days; error rate spikes (Sentry).
- `/api/health` for uptime checks (database round trip).

11. Testing strategy

Level	Scope	Tooling
Unit	Money parsing/rounding, interest accrual, schedules, due dates, crypto, passwords	Vitest (in repo)
Integration	Deposit → ledger, concurrent withdrawals, repayment allocation, arrears job, webhook replay	Vitest + Postgres service container in CI
End-to-end	Register → KYC → deposit → withdraw → loan → repay, on mobile viewport	Playwright
Financial reconciliation	100 sample wallets vs independent spreadsheet	Finance team, UAT
Security	OWASP ZAP baseline in CI; external penetration test	Pre-launch
Load	200 concurrent members on dashboard and deposit webhooks	k6